



Cross-Platform Mobile Challenge (Intern)

Goal: Ship a tiny, offline-first field app that an Operator can actually use on the shop floor to capture downtime and maintenance, and a supervisor can review/acknowledge alerts. Your choices (state management, navigation, storage, sync strategy, UI) matter more than raw features.

Tech: React Native (Expo allowed) or Flutter. Android target required; iOS optional.

What to Build (MVP)

1. Login (mocked)

- Accept any email; persist a mock JWT (string).
- Roles: **operator** or **supervisor** (pick via a simple toggle on first login).

2. Home

- List machines (min 3). Show status chip (RUN/IDLE/OFF).
- Tap → Machine screen.

3. Downtime (Operator)

- “Start/End downtime” + choose a **reason** from a tree (2 levels).
- Optional photo attachment (compress ≤ 200 KB).
- Works **offline**: queue entries and **auto-sync** later.

4. Maintenance (Operator)

- Due/Overdue/Done checklist items per machine (time- or usage-based; your call).
- Mark complete with notes; offline-capable with sync.

5. Alerts (Supervisor)

- Read an **incoming stream** of alert events and show a panel with acknowledge/clear.
- If you don't stand up a backend, simulate via a local **SSE** or periodic fetch.

6. Minimal Reports

- One summary card per machine for the current shift/day: Active time (hh:mm), Idle time, Count (if you add it), and a quick “Top reasons” list. Format is up to you

Keep it lean; do less, but make it robust. UX clarity matters.



Cross-Platform Mobile Challenge (Intern)

Data Hints (intentionally light)

Use/extend these as you see fit.

- **Machine (seed file)**

```
[
  {"id": "M-101", "name": "Cutter 1", "type": "cutter"},
  {"id": "M-102", "name": "Roller A", "type": "roller"},
  {"id": "M-103", "name": "Packing West", "type": "packer"}
]
```

- **Reason tree (seed file)**

```
[
  {"code": "NO-ORDER", "label": "No Order", "children": [{"code": "PLANNED", "label": "Planned"}, {"code": "UNPLANNED", "label": "Unplanned"}]},

  {"code": "POWER", "label": "Power", "children": [{"code": "GRID", "label": "Grid"}, {"code": "INTERNAL", "label": "Internal"}]},

  {"code": "CHANGEOVER", "label": "Changeover", "children": [{"code": "TOOLING", "label": "Tooling"}, {"code": "PRODUCT", "label": "Product"}]}
]
```

- **Alert event (stream record)**

```
{
  "ts": "2025-12-25T09:15:07Z",
  "machine_id": "M-101",
  "rule": "idle_gt_30min",
  "severity": "high",
  "msg": "Idle > 30 min",
}
```



Cross-Platform Mobile Challenge (Intern)

```
"id": "evt-7f3a...9c"  
}
```

You decide how to store offline (SQLite/Room/WatermelonDB/AsyncStorage/SharedPreferences/etc.) and how to sync (push on reconnect, backoff, idempotency by `id/ts`, conflict rules).

Hard Requirements

- **Offline-first:** Put device in airplane mode, capture ≥ 2 downtime events and 1 maintenance update, then reconnect and watch them sync automatically. Show a small queue/badge (e.g., “3 pending”).
- **Acknowledgement flow:** Alert moves through Created $\rightarrow$ Acknowledged $\rightarrow$ Cleared; persist who did it (email) and when.
- **One exported build:** APK (Android), or Expo/Flutter web build link if APK isn’t possible.
- **Security placeholder:** Include a `tenant_id` string everywhere you store or send data (no real auth needed).

What to Submit

- **Live artifact:** APK/Expo/Flutter web link.
- **Repo:** GitHub link. Include a short `README.md` with:
 - “How to run” (dev + prod)
 - Offline & sync design (2–5 bullets)
 - State management choice (and why)
 - What you’d ship next (2–3 bullets)
- **90-second Loom:** Walk through offline capture $\rightarrow$ sync $\rightarrow$ alert ack.
- **(Optional)** If you mocked a stream: include your tiny **SSE** script or JSON timer (≤ 50 lines is fine).



Cross-Platform Mobile Challenge (Intern)

Evaluation (score out of 100)

1. **Offline reliability:** survives app kill/airplane mode; resumes and syncs correctly.
2. **Sync design:** queueing, backoff/jitter, idempotency, conflict choice (last-write/who-wins).
3. **Mobile UX:** clarity, touch targets, empty/error states, latency feedback.
4. **State management:** clean model (e.g., Redux/Zustand/Bloc/Provider), minimal boilerplate.
5. **Data modeling:** sensible local schema (events, checklists, alerts), `tenant_id` carried through.
6. **Code quality:** structure, naming, modularity, small components.
7. **Performance:** fast first render; smooth list/scroll; no obvious jank.
8. **Device features:** photo capture + compression; permission handling.
9. **Build & release:** reproducible run/build steps; APK or Expo link works.
10. **Product sense:** thoughtful trade-offs; small touches (badges, toasts, pull-to-refresh).

Guardrails (to keep it short)

- Don't build a real backend unless you want to; local mocks are fine.
- No need for OEE math or energy KPIs. Focus on **forms + offline + alerts**.
- Use any UI kit you like. Keep bundle size and performance in mind.

Bonus (tie-breakers)

- **Photo + metadata** watermark (machine_id, ts) on save.
- **Background sync** on connectivity change.
- **Tiny theming** (light/dark) and localization scaffold (EN only is fine).

Good luck!

Tarun Purohit

Founder & CEO, LimelightIT

tarun@limelightit.io